

# Website Analytics Engineering Guide

Implementation, maintenance, build, and public-repository notes

Last changed: July 23, 2026

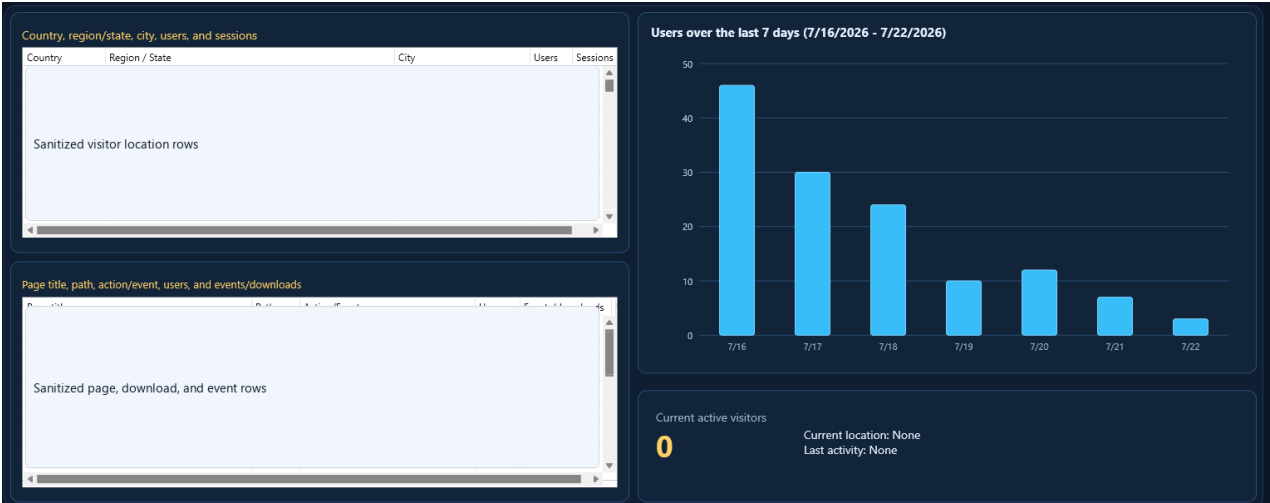
All screenshots in this guide are actual application screenshots that have been cropped and sanitized. Private property names, property IDs, OAuth values, personal paths, and location details are hidden.

## Table of Contents

- Table of Contents.....1
- 1. Engineering Purpose.....2
- 2. Project Structure.....2
- 3. Design Directives for Future Work .....3
- 4. Data Flow .....3
- 5. GA4 Report Responsibilities .....3
- 6. Authentication and Refresh Tokens .....4
- 7. All-Websites Merge Behavior .....4
- 8. SQLite Storage Model .....5
- 9. UI Implementation Notes .....5
- 10. GA4 Request and Parser Notes.....6
- 11. Build and Smoke Test .....7
- 12. Release and Repository Checklist .....7
- 13. Public Repository Hygiene .....7
- 14. Documentation Maintenance.....8
- 15. Maintenance Risk Areas .....8
- 16. Known Limitations and Future Work.....9

# 1. Engineering Purpose

This project is a Delphi FMX desktop application that retrieves selected GA4 data, formats it into in-memory models, and displays it in a clean dashboard. The engineering goal is not to clone GA4. The goal is to make the highest-value website analytics easier to read, while keeping forms editable in RAD Studio and keeping local storage minimal.



Actual dashboard screenshot with private report rows redacted.

# 2. Project Structure

| Folder/file                                   | Role                                                                             |
|-----------------------------------------------|----------------------------------------------------------------------------------|
| WebsiteAnalytics.dpr / .dproj                 | Application entry point and RAD Studio project file.                             |
| WebsiteAnalytics.MainForm.pas/.fmx            | Main dashboard form, hero controls, charts, grids, and realtime display.         |
| WebsiteAnalytics.SettingsForm.pas/.fmx        | Settings and Google sign-in UI.                                                  |
| WebsiteAnalytics.PropertyManagerForm.pas/.fmx | Manage website/GA4 property definitions.                                         |
| WebsiteAnalytics.GA4DataModule.pas            | Builds GA4 API requests, executes HTTP calls, parses report responses.           |
| WebsiteAnalytics.AuthenticationDataModule.pas | Handles desktop OAuth, loopback redirect, access token, and refresh token logic. |
| WebsiteAnalytics.SettingsDataModule.pas       | SQLite-backed settings and schema management.                                    |
| WebsiteAnalytics.Models.pas                   | Memory model types for snapshots, KPI summaries,                                 |

|                          |                                                |
|--------------------------|------------------------------------------------|
|                          | trend points, content rows, and location rows. |
| help                     | Local HTML help used by the Help button.       |
| docs/guides and docs/pdf | Editable guides and companion PDF output.      |

### 3. Design Directives for Future Work

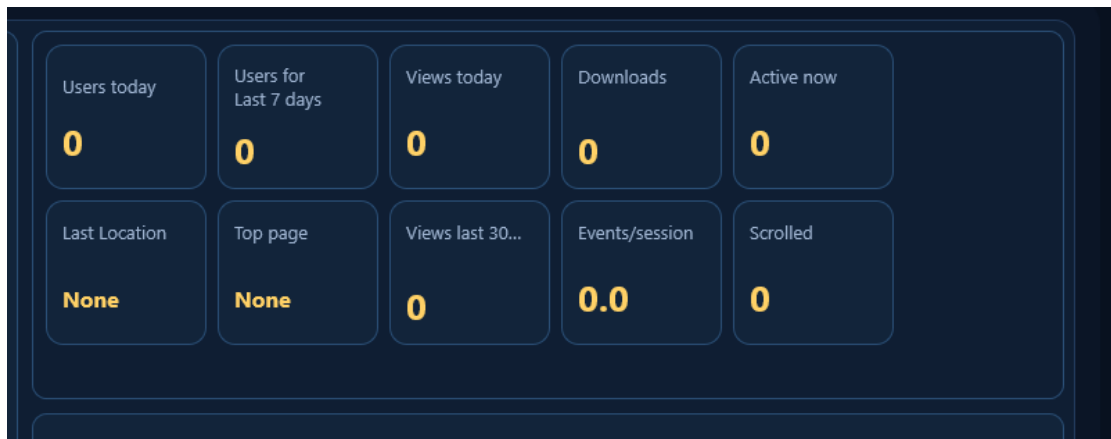
- Forms should remain editable in the RAD Studio IDE.
- Avoid runtime-only UI construction unless it is unavoidable and approved.
- Do not add helper code unless the developer approves it.
- Prefer Object Inspector settings for layout where practical.
- Make a pre-change backup on D: before modifying project files.
- After approved changes are validated, commit and push both the private and public repositories.

### 4. Data Flow

1. The user selects a property and date range in the main form.
2. The main form requests an access token from the authentication data module.
3. The GA4 data module builds runReport or realtime report JSON requests.
4. HTTP responses are parsed into in-memory model objects.
5. The main form updates existing FMX controls: cards, grids, chart paint boxes, and realtime labels.
6. The report snapshot remains in memory and is discarded when the app closes.

### 5. GA4 Report Responsibilities

| Report area     | GA4 data used                                                                                | Displayed as                         |
|-----------------|----------------------------------------------------------------------------------------------|--------------------------------------|
| Summary cards   | Users, views, downloads/events, active users, realtime views, events/session, scrolled users | Ten compact cards.                   |
| Locations       | Country, region, city, users, sessions, engagement                                           | Alphabetized grid.                   |
| Pages/downloads | Page title, path, event/action, users, event/download counts                                 | Bar chart and supporting data logic. |
| Users trend     | Daily users over selected date range                                                         | Bar graph with tooltip.              |
| Realtime        | Active users and realtime location/activity dimensions                                       | Realtime panel.                      |



Actual summary-card area used to expose high-value GA4 metrics.

## 6. Authentication and Refresh Tokens

The app uses Google desktop OAuth. The browser sign-in returns to a loopback redirect URI. The app exchanges the authorization code for tokens. The access token is held in memory. The refresh token is encrypted with Windows DPAPI and saved in the SQLite settings database so startup can reconnect silently.

☒ Retrieve live data when the program opens☒ Compare with the previous period

### GA4 desktop OAuth setup

**OAuth desktop client ID**

OAuth desktop client ID hidden

**OAuth desktop client secret**

OAuth client secret hidden

**Loopback port**

53682

**Redirect URI:** http://127.0.0.1:53682/oauth2redirect

Connect Google opens your browser. Sign in with the Google account that can read these GA4 properties. The app receives a temporary token for this session only; it does not store your Google password or analytics results.

Connect Google

Disconnect

Connection status hidden

Visual theme: dark dashboard with high-contrast cards and readable report panels.

Public-safe storage note: settings local, report rows memory-only

Actual settings area with credentials and status redacted.

- The OAuth client ID and secret belong to the user's own Google Cloud desktop OAuth client.
- The saved refresh token should never be committed or included in a public release.
- Disconnect clears the access token and saved encrypted refresh token.
- Silent reconnect should be attempted before showing Google login screens.

## 7. All-Websites Merge Behavior

All-websites mode is not a separate GA4 property. It is an application-level merge across enabled properties. The app requests each enabled property, aggregates compatible metrics, and then updates the same dashboard controls. This makes the dashboard useful for a broad roll-up, but it also means labels such as Home or /downloads.html may represent the same path on more than one site.

- Additive metrics such as users, sessions, views, events, downloads, and scrolled users can be summed carefully.
- Ratio metrics such as engagement rate and events/session should be recomputed or weighted, not blindly summed.
- Trend points should be merged by date so the chart remains aligned to the selected range.
- Content rows should be grouped by meaningful keys such as page title, path, and event/action.
- Realtime all-sites data can be more expensive and potentially more confusing; individual property view is usually clearer.

## 8. SQLite Storage Model

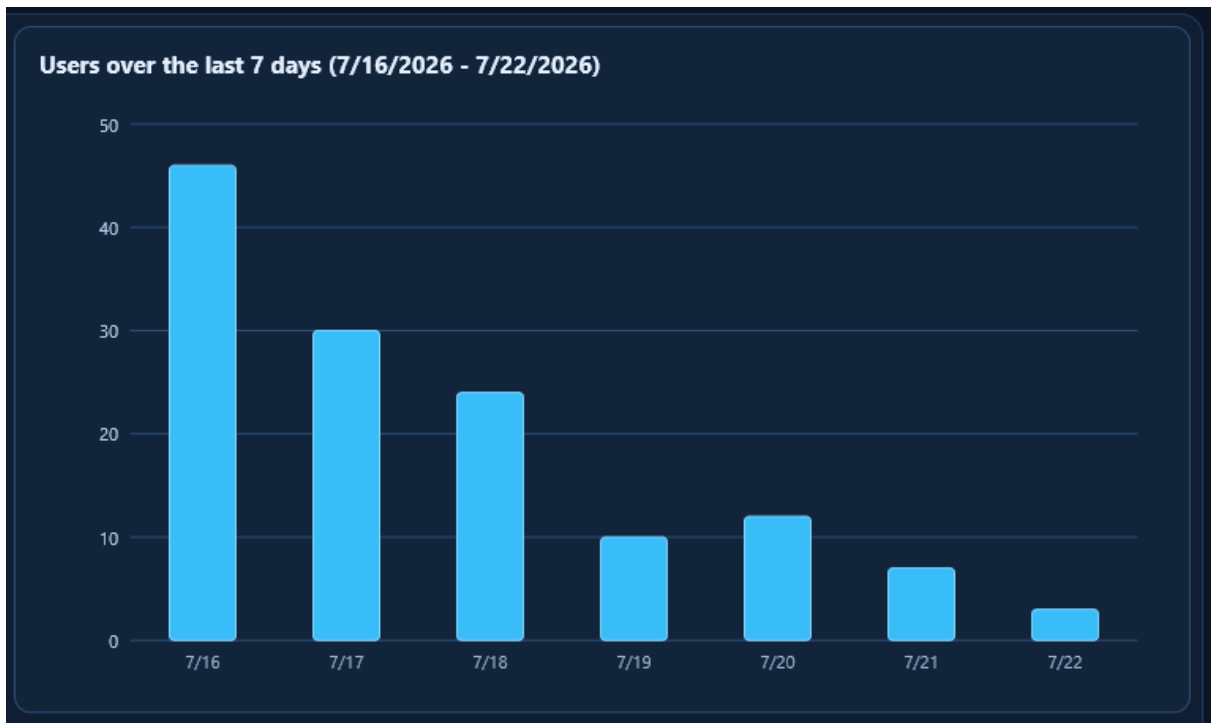
SQLite is used for portable settings, not analytics warehousing. The local database may live beside the source or executable according to the current project setup. The schema SQL is tracked so a fresh database can be created, but real .sqlite3 files should be ignored by Git.

| Data category              | Stored? | Reason                                     |
|----------------------------|---------|--------------------------------------------|
| Default website/date range | Yes     | User preference.                           |
| Property definitions       | Yes     | Needed to query GA4 properties.            |
| Grid widths                | Yes     | Preserve user's manual column adjustments. |
| OAuth client setup         | Yes     | Needed for token exchange.                 |
| Encrypted refresh token    | Yes     | Enables silent reconnect.                  |
| GA4 report rows            | No      | Memory-only design.                        |

## 9. UI Implementation Notes

The main form is intentionally a real FMX form rather than a runtime-built dashboard. Chart painting and data updates are runtime behavior, but control existence, general layout, captions, and editable form structure belong in the designer whenever possible.

- The bar charts are painted in paint boxes from in-memory trend/top-page data.
- Tooltips are compact overlays for exact chart values.
- The grids use FMX string grids and saved column widths.
- Property/date selection and tab changes should trigger appropriate data refresh behavior.
- Avoid code that silently resizes or rebuilds controls unless explicitly approved.



Actual users graph showing a selected date range.

## 10. GA4 Request and Parser Notes

GA4 responses must be parsed defensively. Some successful responses contain no rows. Some dimensions return (not set). Realtime responses may return fewer dimensions than standard runReport responses. The code should treat missing rows as no data rather than as a fatal parser failure.

| Condition                 | Expected engineering behavior                                                            |
|---------------------------|------------------------------------------------------------------------------------------|
| No rows                   | Clear the relevant model list and display zero/None/no rows gracefully.                  |
| Missing rows member       | Treat as empty rather than throwing a user-facing parser error.                          |
| Wrong property ID         | Show the GA4 HTTP error clearly.                                                         |
| Permission denied         | Show enough of the 403 message to guide the user to GA4 Admin access/property ID checks. |
| Token expired             | Attempt one refresh/retry before asking the user to reconnect.                           |
| Realtime location missing | Show None or Current location: None rather than stale                                    |

|  |                |
|--|----------------|
|  | location text. |
|--|----------------|

## 11. Build and Smoke Test

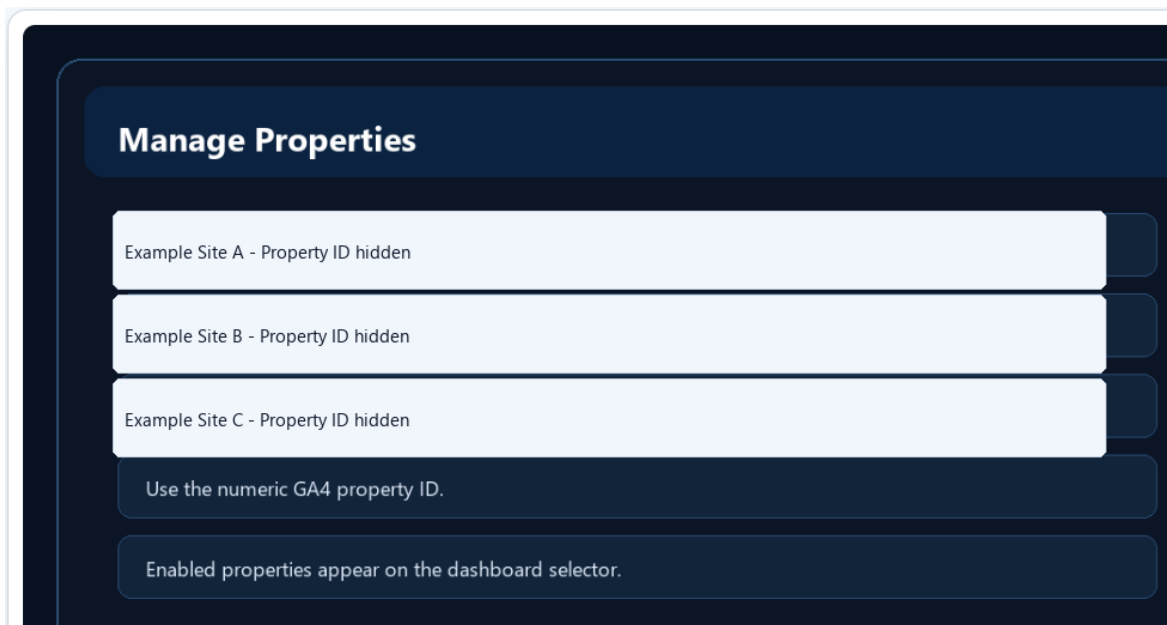
7. Run the RAD Studio environment setup from C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\rsvars.bat.
8. Build Win64 Debug with MSBuild.
9. Build Win32 Debug with MSBuild.
10. Start the executable briefly with startup authentication disabled when appropriate to catch FMX read errors.
11. If the executable is locked, close any running WebsiteAnalytics.exe process and rebuild.

## 12. Release and Repository Checklist

12. Run a private-info scan against documents, help, source text, and tracked configuration files.
13. Verify .gitignore excludes databases/\*.sqlite3 and other local credential material.
14. Build Win64 and Win32.
15. Start the app and confirm the main form loads without FMX property/read errors.
16. Verify help opens from the Help button.
17. Confirm User Guide and Engineering Guide exist as both DOCX and PDF.
18. Review git status and stage only intended files.
19. Commit the private repository.
20. Mirror safe files to the public repository.
21. Commit and push the public repository.

## 13. Public Repository Hygiene

- Never commit real SQLite databases.
- Never commit OAuth secrets, refresh tokens, credential JSON files, PEM files, or .env files.
- Never commit screenshots that show private property IDs, personal locations, or developer-only data.
- Use sanitized real screenshots in documentation.
- Track schema SQL, source, help, guides, and safe image/icon assets.
- Keep private and public repos synchronized only after reviewing the diff for secrets.



*Actual property-management screen with site names and IDs redacted.*

## 14. Documentation Maintenance

Documentation should be rebuilt from actual UI captures and source behavior. Do not use synthetic mockups as screenshots. If an image is cropped from the UI, redact only the private values. The screenshot should still look like the real application.

- User Guide: friendly, procedural, and explanatory.
- Engineering Guide: implementation details, architecture, build process, and safety rules.
- Help: concise but complete local help, with useful cropped screenshots per topic.
- All public screenshots: no personal city, account email, OAuth values, property IDs, or private paths.

## 15. Maintenance Risk Areas

| Area      | Risk                                                                    | Mitigation                                                                       |
|-----------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| OAuth     | Google policy or consent-screen changes can alter the login experience. | Keep setup docs current and test first-run flow periodically.                    |
| GA4 API   | Metric/dimension names can change or behave differently.                | Prefer official GA4 API docs and fail gracefully on missing rows.                |
| FMX grids | Rendering quirks can appear at different sizes/DPI.                     | Avoid unnecessary grid-control churn; test visual changes at common resolutions. |



|                 |                                                   |                                                                                                  |
|-----------------|---------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Public docs     | Screenshots can accidentally reveal private data. | Use actual screenshots, crop to the area discussed, and redact private fields before committing. |
| All-sites merge | Ratios and duplicate labels can mislead.          | Document what is merged and prefer individual property view for diagnosis.                       |

## 16. Known Limitations and Future Work

- GA4 realtime data and standard reports can disagree briefly because they are separate GA4 reporting surfaces.
- Some GA4 dimensions may return (not set). The app should display those rows rather than hiding them.
- The current grid control can have minor FMX rendering quirks. Replacing it should be considered only if the benefit is worth the disruption.
- CSV export, saved report snapshots, and additional graphs are possible future enhancements, but they would change the current memory-only philosophy.